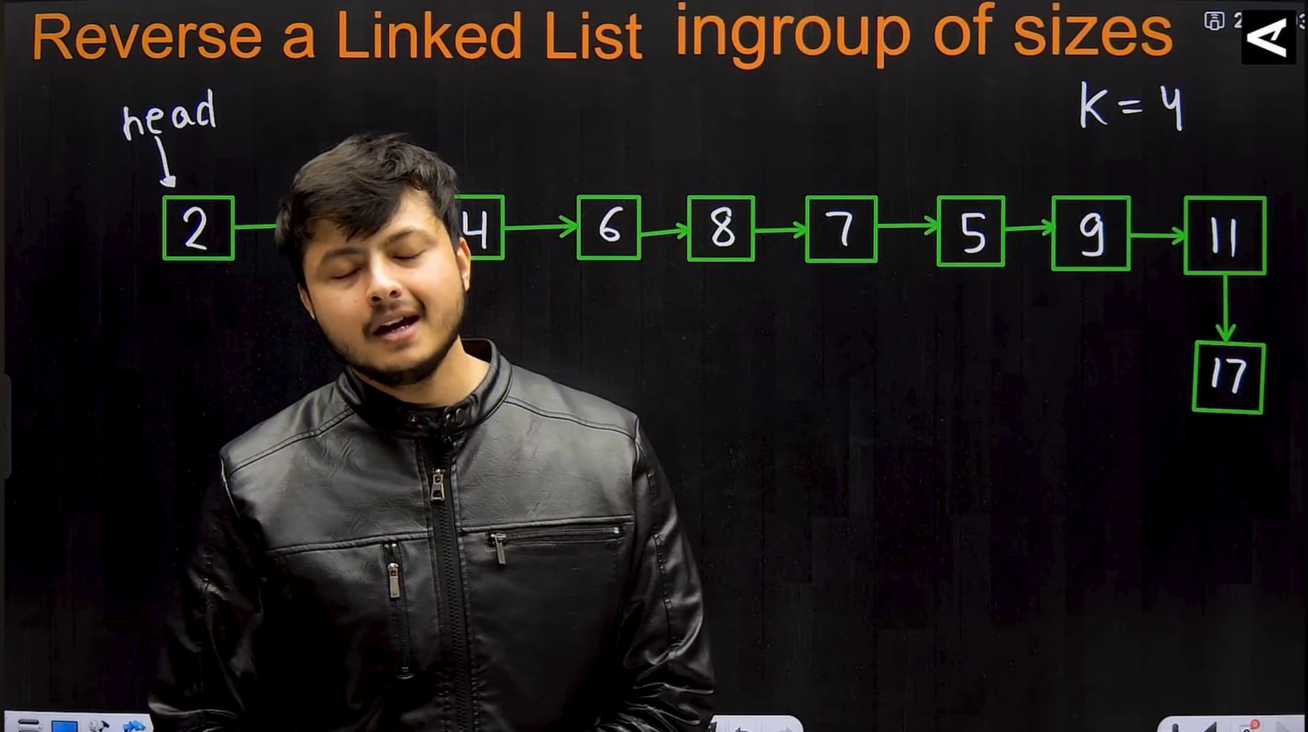
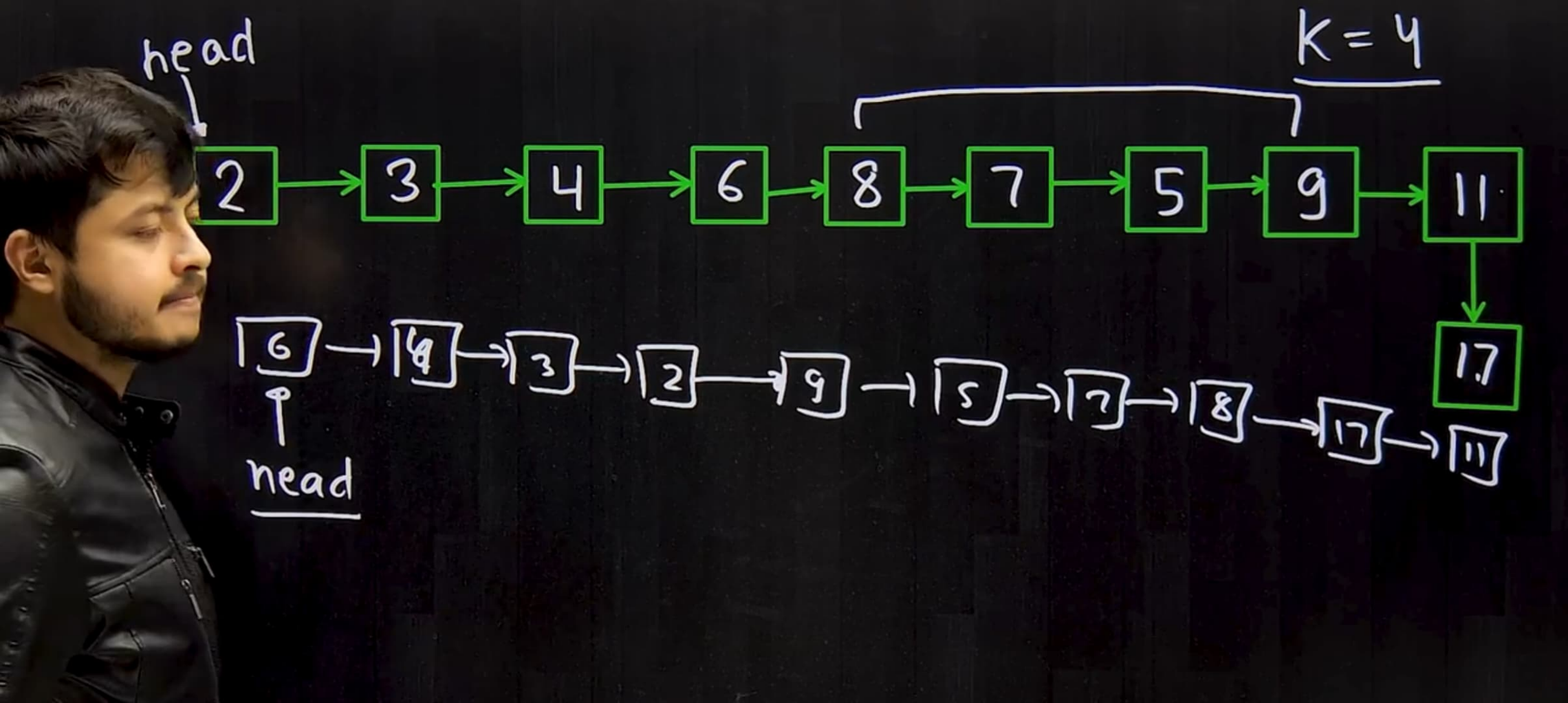
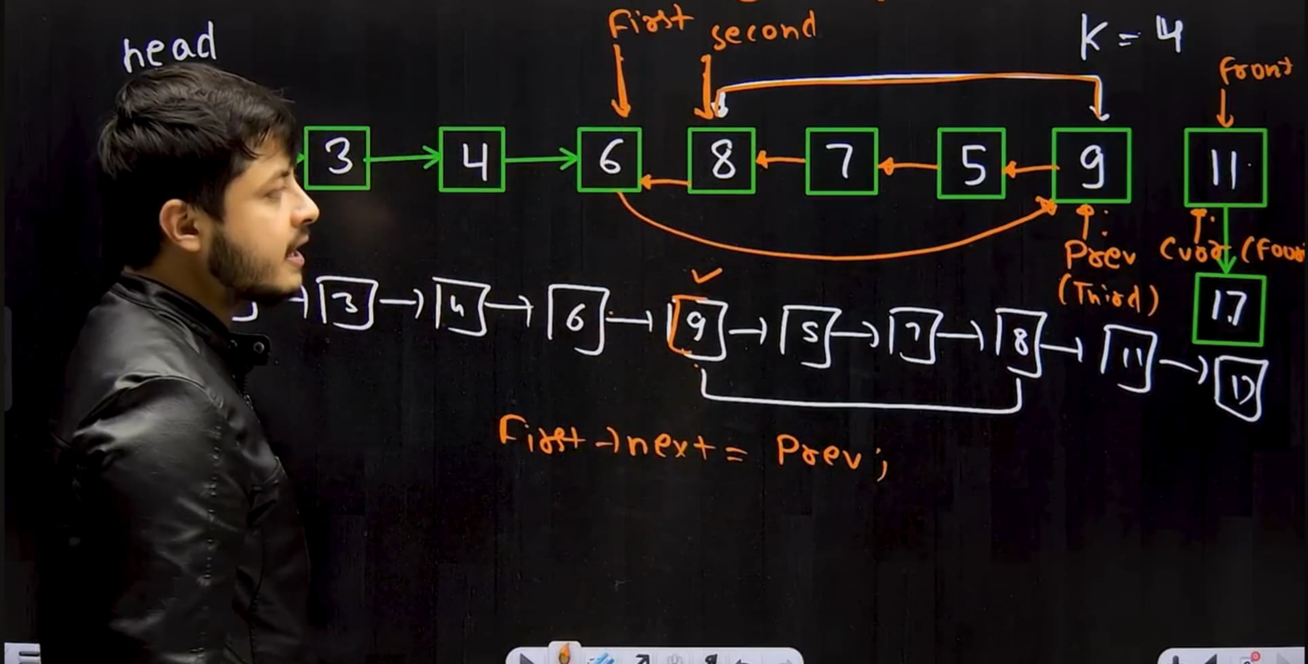
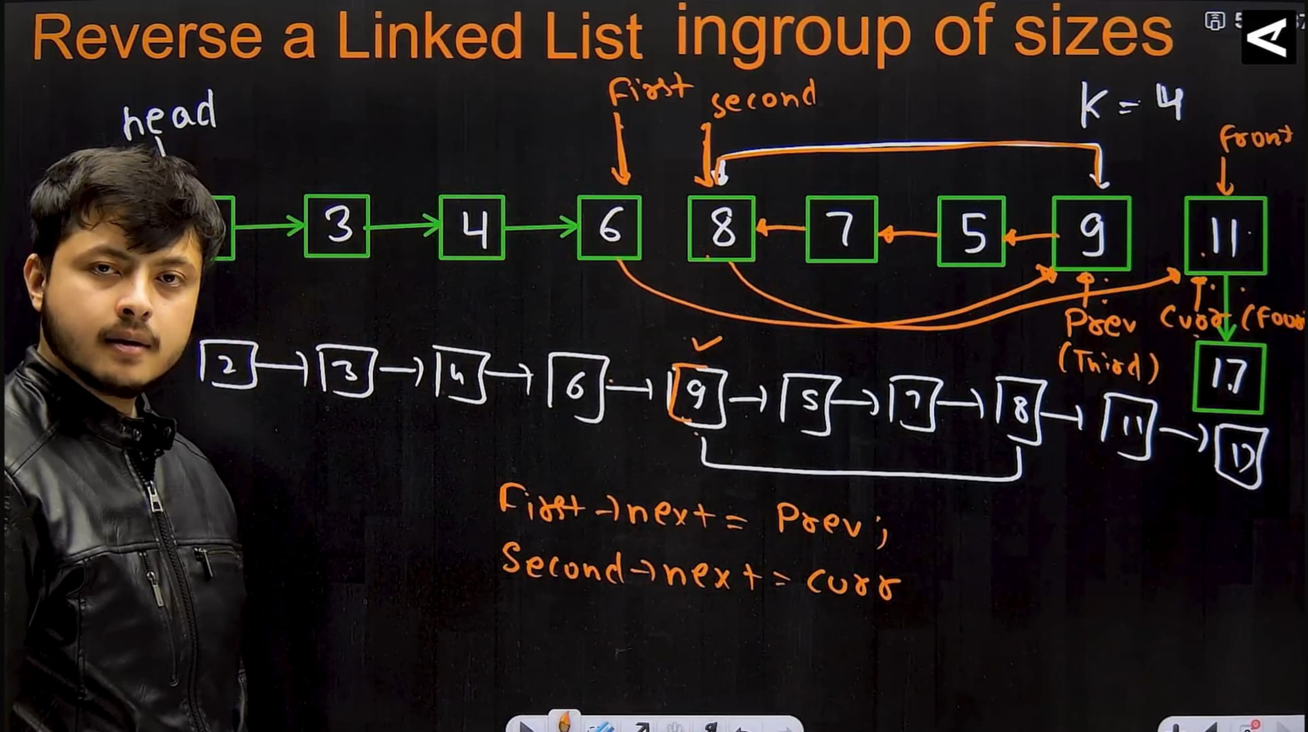

Reverse a Linked List ingroup of sizes
head
K = 4
2
4
6
8
7
5
9
11
17


head
K = 4
2
3
4
6
8
7
5
9
11
17
6
4
3
2
9
5
7
8
17
11
head

Reverse a Linked List ingroup of sizes




Reverse a Linked List ingroup of sizes
head
First
second
K = 4
front
Prev
(Third)
Curr
First->next = Prev;
Second->next = curr

```
Node * Second = first->next;
Node * Prev = First;
Node * Curr = first->next;
while(K)
{
    front = curr->next;
    Curr->next = Prev;
    Prev = Curr;
    Curr = Front;
    K--;
}
first->next = prev;
Second->next = Curr;
```

```
first = ...d ;
...head->next
...te first
...turn head;
```

```
while(first->next)
{
    int x = K;
    Node * second = first->next;
    Node * prev = first;
    Node * curr = first->next;
    while(x && curr)
    {
        front = curr->next;
        curr->next = prev;
        prev = curr;
        curr = front;
        x--;
    }
    first->next = prev;
    second->next = curr;
    first = second;
}
```

```
Node * First=new Node(0);
First→next = head;
head = First;
```

C++ (g++ 5.4) Average Time: 30m Start Timer

Reverse a Linked List in groups of given size

Medium Accuracy: 57.08% Submissions: 192K+ Points: 4

Given a linked list of size **N**. The task is to reverse every **k** nodes (where k is an input to the function) in the linked list. If the number of nodes is not a multiple of *k* then left-out nodes, in the end, should be considered as a group and must be reversed (See Example 2 for clarification).

Example 1:

```
Input:
LinkedList: 1->2->2->4->5->6->7->8
K = 4
Output: 4 2 2 1 8 7 6 5
Explanation:
The first 4 elements 1,2,2,4 are reversed first
and then the next 4 elements 5,6,7,8. Hence, the
resultant linked list is 4->2->2->1->8->7->6->5.
```

Example 2:

```
Input:
LinkedList: 1->2->3->4->5
```

```cpp
public:
struct node *reverseIt (struct node *head, int k)
{

    // Complete this method
    // dummy node create karo
    Node *first = new Node(0);
    first->next = head;
    head = first;

    while(first->next)
    {
        int x = K;
        Node *second = first->next;
        Node *prev =first;
        Node *curr = first->next;
    }
```


Custom Input Compile & Run Submit

Reverse a Linked List in groups of given size

Medium Accuracy: 57.08% Submissions: 192K+ Points: 4

Given a linked list of size **N**. The task is to reverse every **k** nodes (where k is an input to the function) in the linked list. If the number of nodes is not a multiple of *k* then left-out nodes, in the end, should be considered as a group and must be reversed (See Example 2 for clarification).

Example 1:

Input:
LinkedList: 1->2->2->4->5->6->7->8
K = 4
Output: 4 2 2 1 8 7 6 5
Explanation:
The first 4 elements 1,2,2,4 are reversed first and then the next 4 elements 5,6,7,8. Hence, the resultant linked list is 4->2->2->1->8->7->6->5.

Example 2:

Input:
LinkedList: 1->2->3->4->5

```cpp
public:
struct node *reverseIt (struct node *head, int k)
{

    // Complete this method
    // dummy node create karo
    Node *first = new Node(0);
    first->next = head;
    head = first;
    int x;
    Node *second, *prev, *curr, *front;

    while(first->next)
    {
        int x = K;
        second = first->next;
        prev =first;
        curr = first->next;


    }
```

Reverse a Linked List in groups of given size

Medium Accuracy: 57.08% Submissions: 192K+ Points: 4

Given a linked list of size **N**. The task is to reverse every **k** nodes (where k is an input to the function) in the linked list. If the number of nodes is not a multiple of *k* then left-out nodes, in the end, should be considered as a group and must be reversed (See Example 2 for clarification).

Example 1:

Input:
LinkedList: 1->2->2->4->5->6->7->8
K = 4
Output: 4 2 2 1 8 7 6 5
Explanation:
The first 4 elements 1,2,2,4 are reversed first and then the next 4 elements 5,6,7,8. Hence, the resultant linked list is 4->2->2->1->8->7->6->5.

Example 2:

Input:
LinkedList: 1->2->3->4->5

```cpp
Node *first = new Node(0);
first->next = head;
head = first;
int x;
Node *second, *prev, *curr, *front;

while(first->next)
{
    x = K;
    second = first->next;
    prev =first;
    curr = first->next;

    while(x&&curr)
    {
        front = curr->next;
        curr->next = prev;
        prev = curr;
        curr = front;

    }
```

Problem Editorial Submissions Comments

C++ (g++ 5.4) Average Time: 30m Start Timer

Reverse a Linked List in groups of given size

Medium Accuracy: 57.08% Submissions: 192K+ Points: 4

Given a linked list of size **N**. The task is to reverse every **k** nodes (where k is an input to the function) in the linked list. If the number of nodes is not a multiple of *k* then left-out nodes, in the end, should be considered as a group and must be reversed (See Example 2 for clarification).

Example 1:

Input:
LinkedList: 1->2->2->4->5->6->7->8
K = 4
Output: 4 2 2 1 8 7 6 5
Explanation:
The first 4 elements 1,2,2,4 are reversed first and then the next 4 elements 5,6,7,8. Hence, the resultant linked list is 4->2->2->1->8->7->6->5.

Example 2:

Input:
LinkedList: 1->2->3->4->5

```
        second = first->next;
        prev =first;
        curr = first->next;

        while(x&&curr)
        {
            front = curr->next;
            curr->next = prev;
            prev = curr;
            curr = front;
            x--;
        }

        first->next = prev;
        second->next = curr;
        first = second;

    }
```


Custom Input Compile & Run Submit

group and must be reversed (See Example 2 for clarification).

Example 1:

Input:
LinkedList: 1->2->2->4->5->6->7->8
K = 4
Output: 4 2 2 1 8 7 6 5
Explanation:
The first 4 elements 1,2,2,4 are reversed first and then the next 4 elements 5,6,7,8. Hence, the resultant linked list is 4->2->2->1->8->7->6->5.

Example 2:

Input:
LinkedList: 1->2->3->4->5
K = 3
Output: 3 2 1 5 4
Explanation:
The first 3 elements are 1,2,3 are reversed first and then elements 4,5 are reversed.Hence, the resultant linked list is 3->2->1->5->4.

Your Task:
You don't need to read input or print anything. Your task is to complete the function **reverse**() which should reverse the linked list in group of size **k** and

```
                front = curr->next;
                curr->next = prev;
                prev = curr;
                curr = front;
                x--;
            }

            first->next = prev;
            second->next = curr;
            first = second;
        }

        // Dummy Node create delete
        first = head;
        head = head->next;
        delete first;

        return head;
```


Custom Input Compile & Run Submit

Reverse LL in groups of size K :

K = 4

2 → 3 → 4 → 6 → 8 → 7 → 5 → 9 → 11 → 17

6 → 4 → 3 → 2 → 9 → 5 → 7 → 8 → 17 → 11 .

Subproblem :

2 → 3 → 4 → 6 → 9 → 5 → 7 → 8 → 11 → 17

O/P ?

K = 4

first

2 → 3 → 4 → 6 → 8 → 7 → 5 → 9 → 11 → 17

prev

while (first → next) {

```
Node* second = first → next
Node* prev = first
Node* curr = first → next

while (K && curr) {
    Node fut = curr → next
    curr → next = prev
    prev = curr
    curr = fut
    K--
}
```

K == 0 Stop.

$\boxed{\text{first → next = prev} \quad \text{second → next = curr}}$

first = ~~prev~~ second * carefully

}

first
second

~~At end:~~
~~first → next = prev~~
~~second → next = NULL~~

Don't have to, not